Clinic Management System

DBMS Mini-Project Submission

Database Management Systems | Semester 6 | S26

Group 23

bt23cse012 · bt23cse002 · bt23cse007

Technology Stack

Database:	Oracle 19c + VPD (DBMS_RLS)
Backend:	Python 3 + FastAPI + JWT Authentication
Frontend:	React + TypeScript
Scale:	24 Entities	9 Triggers	4 Views	13 Indexes

BONUS: VPD Row-Level Security | OTP Auth | Emergency System | Pharmacy Module | Lab Module | Family Links

Table of Contents

1. Compliance Checklist — Point-by-Point Verification

2. Project Overview & Entities (All 24)

3. Entity-Relationship (ER) Diagram

4. Functional Dependencies — Complete Catalogue

5. Normalisation — Step-by-Step (1NF → 2NF → 3NF → BCNF)

6. Trigger Catalogue

7. VPD Design (Bonus)

8. Authentication Architecture (Bonus)

9. API Route Summary

10. Views, Materialized Views & Indexes

11. Bonus Features Summary

1. Compliance Checklist — Point-by-Point Verification

The table below maps each deliverable from the problem statement to the corresponding implementation in this project. Every mandatory requirement is fully satisfied; bonus items are starred (★).

Status	Requirement	Evidence in Project
✓ YES	**Phase 1 – ER Diagram (entities, relationships, cardinalities)**	Full ER diagram with 24 entities, all cardinalities, weak entity double-borders. See Section 3.
✓ YES	**Phase 1 – Relational Schema with PKs and FKs**	24 tables; every FK explicitly declared with ON DELETE constraints; surrogate PKs via GENERATED ALWAYS AS IDENTITY.
✓ YES	**Phase 1 – Normalisation to 3NF (with BCNF discussion)**	All tables proven to 3NF; BCNF analysis in Section 5. Deliberate documented denormalizations for VPD performance.
✓ YES	**Phase 2 – DDL scripts with NOT NULL / UNIQUE / CHECK / FK constraints**	CK_BILLING_TAX enforces 18% GST; UQ_APPOINTMENT_SLOT prevents double-booking; CK_EMERGENCY_SEVERITY limits severity values.
✓ YES	**Phase 2 – Trigger: enforce business rule (prevent double-booking)**	trg_doctor_leave_block_booking raises ORA-20010 when new appointment overlaps approved doctor leave window.
✓ YES	**Phase 2 – Trigger: automate state change (payment status)**	TRG_BILLING_STATUS auto-sets PENDING / PARTIAL / PAID based on amount_paid vs total_amount on every BILLING UPDATE.
✓ YES	**Phase 2 – Minimum 2 triggers**	9 triggers implemented (see Section 6). Exceeds minimum by 7.
✓ YES	**Phase 3 – REST API: patient registration**	PUT /api/patient/profile/complete — completes patient registration; OTP-based login via phone number.
✓ YES	**Phase 3 – REST API: appointment booking**	POST /api/patient/appointments — books appointment; trigger auto-creates BILLING row with snapshotted fee.
✓ YES	**Phase 3 – REST API: prescription creation**	POST /api/doctor/appointments/{id}/prescription — issues prescription with medication line items.
✓ YES	**Phase 3 – REST API: billing report**	GET /api/admin/revenue-report — refreshes materialized view V_DEPT_REVENUE and returns per-department monthly revenue.
✓ YES	**Phase 4 – React SPA: data entry forms**	Appointment booking, patient registration, prescription issuance, and emergency submission forms.
✓ YES	**Phase 4 – Tabular data view**	Appointment schedule tables in all three portals; billing outstanding view; patient list.
✓ YES	**Phase 4 – Chart or dashboard widget**	Admin dashboard KPI cards (total patients, today appointments, revenue, pending bills); V_DEPT_REVENUE chart.
✓ YES	**Phase 4 – No direct DB access from browser**	All data flows through FastAPI backend; Oracle credentials never exposed to the frontend.
✓ YES	**Stack: Oracle database**	Oracle 19c.
✓ YES	**Stack: Python backend**	FastAPI (Python 3).

✓ YES	**Stack: React frontend**	React + TypeScript.
★ BONUS	**VPD / Row-Level Security**	DBMS_RLS applied to 10 tables; per-role predicates enforced at DB layer.
★ BONUS	**OTP Authentication for patients**	Phone → 6-digit OTP (hashed, TTL 10 min); JWT issued on verify.
★ BONUS	**JWT with Token Blacklist**	TOKEN_BLACKLIST table; middleware rejects revoked tokens on every request.
★ BONUS	**Materialized View (V_DEPT_REVENUE)**	DBMS_MVIEW.REFRESH on demand; powers admin revenue chart.
★ BONUS	**AUDIT_LOG with JSON CLOBs**	TRG_AUDIT_PATIENT stores old/new JSON snapshots for every DML on PATIENT.
★ BONUS	**Lab Test Module (Task 18)**	LAB_TEST_CATALOGUE, LAB_ORDER, LAB_RESULT, VITAL_SIGNS — full workflow.
★ BONUS	**Pharmacy & Dispensing Module (Task 18)**	PHARMACY_ITEM inventory + DISPENSING table; trigger deducts stock on dispense.
★ BONUS	**Emergency System (Task 19)**	EMERGENCY_REQUEST with severity levels; trg_emergency_fanout notifies doctors.
★ BONUS	**Doctor Leave & Notifications (Task 19)**	DOCTOR_LEAVE + booking-block trigger; DOCTOR_NOTIFICATION + PATIENT_NOTIFICATION.
★ BONUS	**Family Link (Task 19)**	PATIENT_FAMILY_LINK — bidirectional M:N self-relationship with 5 relationship types.

2. Project Overview & Complete Entity Catalogue (24 Entities)

The Clinic Management System models the full lifecycle of a patient interaction — from registration and appointment booking through clinical consultation, lab testing, pharmacy dispensing, and billing. Oracle's Virtual Private Database (VPD) enforces row-level security: doctors see only their patients, patients see only their own records, and the administrator has unrestricted access.

2.1 Core / Structural Entities

#	Entity	Description
1	**DEPARTMENT**	Clinical divisions (Cardiology, Orthopaedics …) with an optional head-of-department doctor (self-referencing FK).
2	**DOCTOR**	Clinical staff: qualifications, specialisation, consultation fee, department assignment.
3	**STAFF**	Non-clinical personnel (receptionist, nurse, lab tech, pharmacist) assigned to a department.
4	**PATIENT**	Demographic and contact details; one-to-one link with MEDICAL_RECORD.
5	**MEDICAL_RECORD**	Long-term health summary: allergies, chronic conditions, surgical history, family history, vaccination records.
6	**CLINIC_USER**	Authentication table; exists for every portal login; linked to DOCTOR or PATIENT via linked_entity_id (polymorphic).
7	**TOKEN_BLACKLIST** ■ weak	Revoked JWTs; middleware rejects blacklisted tokens — prevents token reuse after logout.

2.2 Transactional Entities

#	Entity	Description
8	**APPOINTMENT**	A 30-minute consultation slot; auto-creates a BILLING row on insert via TRG_CREATE_BILLING.
9	**CONSULTATION**	Doctor's clinical notes (chief complaint, diagnosis, treatment notes) — one per appointment.
10	**PRESCRIPTION**	Header for medication orders tied to one consultation.
11	**PRESCRIPTION_ITEM** ■ weak	Individual medication line items: dosage, frequency, duration, quantity. Existence depends on PRESCRIPTION.
12	**BILLING**	Financial ledger; total_amount is a virtual computed column = (fee + extra) × (1 – discount%) × (1 + 18% GST).
13	**AUDIT_LOG** ■ weak	Immutable DML audit trail storing old/new values as JSON CLOBs — written by triggers.

2.3 Extended Clinical Entities (Task 18 — Bonus)

#	Entity	Description
14	**LAB_TEST_CATALOGUE**	Master list of 20 diagnostic tests with pricing and turnaround times.
15	**LAB_ORDER**	Doctor raises a test request within a consultation; status progresses PENDING → COMPLETED.

16	**LAB_RESULT** ■ weak	Result uploaded per order; is_abnormal flag surfaces priority items; existence depends on LAB_ORDER.
17	**VITAL_SIGNS** ■ weak	Pre-consultation vitals: BP (systolic/diastolic), heart rate, temperature, SpO■, weight, height.
18	**PHARMACY_ITEM**	Clinic drug inventory with batch number, expiry date, reorder threshold, and stock quantity.
19	**DISPENSING**	Medicines dispensed from pharmacy per prescription; trigger deducts stock on every insert.

2.4 Task 19 Entities (Bonus)

#	Entity	Description
20	**PATIENT_FAMILY_LINK** ■ weak	Bidirectional M:N family relationships between two PATIENT records (SPOUSE, PARENT, CHILD, SIBLING, GUARDIAN).
21	**DOCTOR_LEAVE** ■ weak	Doctor unavailability windows; booking trigger rejects appointments overlapping an approved leave window.
22	**EMERGENCY_REQUEST**	Patient-raised emergency with severity (LOW/MODERATE/HIGH/CRITICAL), location, and description; admin assigns doctor.
23	**DOCTOR_NOTIFICATION** ■ weak	In-app alerts to doctors: appointment cancellations, critical emergency assignments.
24	**PATIENT_NOTIFICATION** ■ weak	In-app alerts to patients: lab results ready, appointment status changes.

3. Entity-Relationship (ER) Diagram

The diagram below uses standard ER notation: rectangles for strong entities, double-bordered rectangles for weak entities, diamonds for relationships, with crow's-foot cardinality labels on each connecting line. All 24 entities and their relationships are shown. Dashed lines indicate derived/authentication links.

Figure 1 — Clinic Management System ER Diagram (all 24 entities)

3.1 Relationship Summary

Relationship	Type	Description
DEPARTMENT — DOCTOR	1 : N	A department has many doctors; each doctor belongs to one department
DEPARTMENT — DOCTOR (head)	0..1:1	A department may designate one doctor as head (self-referencing via head_doctor_i
DEPARTMENT — STAFF	1 : N	Support staff belong to exactly one department
DOCTOR — APPOINTMENT	1 : N	A doctor handles many appointments
PATIENT — APPOINTMENT	1 : N	A patient may book many appointments
APPOINTMENT — BILLING	1 : 1	Each appointment generates exactly one invoice (auto via trigger)
APPOINTMENT — CONSULTATION	1 : 0..1	At most one clinical note per appointment
CONSULTATION — VITAL_SIGNS	1 : 0..1	One vitals set recorded per consultation
CONSULTATION — PRESCRIPTION	1 : 0..1	At most one prescription per consultation
PRESCRIPTION — PRESCRIPTION_ITEM	1 : N	A prescription has one or more medication lines
CONSULTATION — LAB_ORDER	1 : N	Multiple tests can be ordered per consultation
LAB_ORDER — LAB_RESULT	1 : 0..1	One result upload per order
LAB_ORDER — LAB_TEST_CATALOGUE	N : 1	Each order references one test from the master catalogue
PRESCRIPTION — DISPENSING	1 : N	Multiple items dispensed from pharmacy per prescription
PHARMACY_ITEM — DISPENSING	1 : N	One inventory item can be dispensed many times across prescriptions
PATIENT — MEDICAL_RECORD	1 : 1	One permanent health summary per patient
PATIENT — PATIENT (via link)	M : N	Family relationships via PATIENT_FAMILY_LINK (weak associative entity)
DOCTOR — DOCTOR_LEAVE	1 : N	A doctor may declare multiple leave windows
PATIENT — EMERGENCY_REQUEST	1 : N	A patient may raise multiple emergencies
DOCTOR — DOCTOR_NOTIFICATION	1 : N	Multiple alerts may be queued for a doctor
PATIENT — PATIENT_NOTIFICATION	1 : N	Multiple alerts may be queued for a patient
CLINIC_USER — DOCTOR/PATIENT	0..1:1	Polymorphic auth link via linked_entity_id (one user per staff/patient)

4. Functional Dependencies — Complete Catalogue

For every table the primary key is shown in BOLD. Candidate keys are marked (CK). Derived/virtual columns are shown with a note. All FDs were enumerated before normalisation to drive the 1NF→2NF→3NF decomposition in Section 5.

FDs for DEPARTMENT

```
department_id → name, description, location, head_doctor_id
name → department_id (CK: name is unique)
```

head_doctor_id is a nullable FK back to DOCTOR (self-referencing for department head).

FDs for DOCTOR

```
doctor_id → employee_id, full_name, specialisation, qualification,
years_of_experience, contact, email, consultation_fee, department_id
employee_id → doctor_id (CK)
email → doctor_id (CK)
```

Three candidate keys: doctor_id (surrogate), employee_id, email.

FDs for STAFF

```
staff_id → employee_id, full_name, role, contact, email, department_id
employee_id → staff_id (CK)
email → staff_id (CK)
```

FDs for PATIENT

```
patient_id → full_name, dob, gender, blood_group, contact_number,
email, address, emergency_contact_name,
emergency_contact_phone, insurance_provider, insurance_number
email → patient_id (CK)
```

FDs for MEDICAL_RECORD

```
record_id → patient_id, allergies, chronic_conditions,
surgical_history, family_history, vaccination_records
patient_id → record_id (CK — one record per patient)
```

FDs for CLINIC_USER

```
user_id → username, password_hash, role, linked_entity_id, is_active
username → user_id (CK)
linked_entity_id → user_id (CK — one login per doctor/patient)
```

FDs for TOKEN_BLACKLIST

```
token_id → token_hash, revoked_at, user_id
token_hash → token_id (CK — hash of JWT is unique)
```

Weak entity; meaningful only in the context of CLINIC_USER.

FDs for APPOINTMENT

```
appointment_id → patient_id, doctor_id, department_id,
appt_date, slot_start, slot_end, status
(doctor_id, appt_date, slot_start) → appointment_id (CK — unique slot per doctor)
```

department_id stored here as a documented denormalization (explained in Section 5).

FDs for CONSULTATION

```
consultation_id → appointment_id, chief_complaint, diagnosis,
treatment_notes, follow_up_date
appointment_id → consultation_id (CK — one consult per appointment)
```

FDs for PRESCRIPTION

```
prescription_id → consultation_id, prescribed_date, notes
consultation_id → prescription_id (CK — one prescription per consultation)
```

FDs for PRESCRIPTION_ITEM

```
item_id → prescription_id, medication_name,
dosage, frequency, duration_days, quantity
```

Weak entity — item_id is meaningful only within its PRESCRIPTION.

FDs for BILLING

```
billing_id → appointment_id, consultation_fee, additional_charges,
discount_pct, tax_pct, total_amount (virtual),
amount_paid, payment_mode, payment_status
appointment_id → billing_id (CK)
{consultation_fee, additional_charges, discount_pct, tax_pct} → total_amount
(functional derivation — virtual column)
```

total_amount is a computed virtual column; not independently stored. consultation_fee is SNAPSHOTTED at booking time (see Section 5 for 3NF rationale).

FDs for LAB_TEST_CATALOGUE

```
lab_test_id → test_name, category, price, turnaround_days
test_name → lab_test_id (CK)
```

FDs for LAB_ORDER

```
lab_order_id → consultation_id, patient_id, lab_test_id,
ordered_by_doctor_id, status, priority, clinical_notes
```

patient_id stored as documented denorm for VPD efficiency (transitive path would be lab_order→consultation→appointment→patient).

FDs for LAB_RESULT

```
result_id → lab_order_id, result_text, result_summary,
is_abnormal, reviewed_by_doctor_id, result_date
lab_order_id → result_id (CK — one result per order)
```

Weak entity — existence depends entirely on LAB_ORDER.

FDs for VITAL_SIGNS

```
vital_id → consultation_id, patient_id, bp_systolic,
bp_diastolic, heart_rate, temperature, weight_kg,
height_cm, spo2, respiratory_rate, notes
consultation_id → vital_id (CK)
```

patient_id stored as documented denorm for VPD filter efficiency.

FDs for PHARMACY_ITEM

```
pharmacy_item_id → drug_name, generic_name, batch_number, expiry_date,
stock_quantity, reorder_threshold, unit_price,
is_active, is_deleted
batch_number → pharmacy_item_id (CK)
```

FDs for DISPENSING

```
dispensing_id → prescription_id, pharmacy_item_id, patient_id,
quantity_dispensed, dispensed_at, dispensed_by_staff_id
```

patient_id stored as documented denorm for VPD filter efficiency.

FDs for DOCTOR_LEAVE

```
leave_id → doctor_id, start_datetime, end_datetime,
reason, status (PENDING/APPROVED/REJECTED)
```

Weak entity — meaningful only in context of its DOCTOR.

FDs for EMERGENCY_REQUEST

```
emergency_id → patient_id, severity, description, location,
status, assigned_doctor_id, created_at, resolved_at
```

FDs for DOCTOR_NOTIFICATION

```
notification_id → doctor_id, message, notification_type, is_read, created_at
```

Weak entity — existence depends on DOCTOR.

FDs for PATIENT_NOTIFICATION

```
notification_id → patient_id, message, notification_type, is_read, created_at
```

Weak entity — existence depends on PATIENT.

FDs for PATIENT_FAMILY_LINK

```
link_id → patient_id_1, patient_id_2,
relationship_type (SPOUSE/PARENT/CHILD/SIBLING/GUARDIAN)
```

Weak associative entity capturing M:N self-relationship on PATIENT.

FDs for AUDIT_LOG

```
audit_id → table_name, operation (INSERT/UPDATE/DELETE),
record_id, old_values (CLOB JSON), new_values (CLOB JSON),
changed_by, changed_at
```

Weak entity written exclusively by triggers; no application DML.

5. Normalisation — Step-by-Step Proof (1NF → 2NF → 3NF → BCNF)

5.1 First Normal Form (1NF)

Definition: Every attribute is atomic (no repeating groups or multi-valued attributes); every row is uniquely identifiable.

Starting Point — Conceptual Unnormalised Relation (UNR)

Before schema design, a naive single-table model was considered (for analysis only):

```
CLINIC_UNR( patient_id, patient_name, patient_phone, appt_date, doctor_name, doctor_dept,
medications_list[], -- MULTI-VALUED: violates 1NF diagnosis, treatment, amount_due, payment_status )
```

1NF Violations identified:

- medications_list[] is a repeating, multi-valued attribute — one appointment can have many medications.
- diagnosis and treatment can relate to multiple conditions (conceptually repeating).
- doctor_name and doctor_dept embed doctor data into every appointment row — redundancy.

1NF Decomposition — Actions Taken

To achieve 1NF, we decomposed the UNR into separate single-valued tables:

Action	Result
Extract medications list into a separate relation	PRESCRIPTION_ITEM table — one row per medication per prescription
Extract doctor data into its own relation	DOCTOR table — each doctor appears once; appointment references by FK
Separate appointment from clinical notes	CONSULTATION table — one row per visit note, linked to APPOINTMENT
Add surrogate PKs to all tables	GENERATED ALWAYS AS IDENTITY — guarantees row uniqueness
Store allergies/conditions as single VARCHAR2 text value	MEDICAL_RECORD — pragmatic; each cell still holds one atomic value

Result: After 1NF decomposition we have 10+ tables, all with atomic attributes and unique primary keys. No table contains multi-valued or repeating attributes.

5.2 Second Normal Form (2NF)

Definition: In 1NF AND every non-key attribute is fully functionally dependent on the *entire* primary key (no partial dependencies). Partial dependencies only arise with composite primary keys.

Potential 2NF Violation — PRESCRIPTION_ITEM with composite PK

A naïve design considered (prescription_id, medication_name) as a composite PK for PRESCRIPTION_ITEM:

```
PRESCRIPTION_ITEM_NAIVE(prescription_id, medication_name, dosage, frequency, duration, unit_price) FD
Analysis: {prescription_id, medication_name} → dosage, frequency, duration (FULL dependency — OK)
medication_name → unit_price (PARTIAL — violates 2NF!) unit_price depends only on medication_name, not
on the full composite key.
```

Problem: If the unit price of a drug changes, every PRESCRIPTION_ITEM row for that drug must be updated — update anomaly.

2NF Decomposition Applied

```
PRESCRIPTION_ITEM(item_id PK, prescription_id FK, medication_name, dosage, frequency, duration_days,
quantity) -- item_id is surrogate PK; unit_price NOT stored here PHARMACY_ITEM(pharmacy_item_id PK,
drug_name, unit_price, stock_quantity, ...) -- pricing lives in the drug catalogue; referenced at
dispensing time
```

Resolution: By using a surrogate PK (item_id) for PRESCRIPTION_ITEM, partial dependency is structurally impossible. Unit prices are kept in PHARMACY_ITEM (the drug master). This eliminates the partial dependency and prevents price update anomalies.

All other tables use single-attribute surrogate PKs, so partial dependency cannot exist structurally — they are trivially in 2NF by design.

5.3 Third Normal Form (3NF)

Definition: In 2NF AND every non-key attribute is non-transitively dependent on the primary key — no non-key attribute determines another non-key attribute.

3NF Analysis — Key Design Decisions

Decision 1: BILLING — Snapshot consultation_fee (prevents transitive FD)

```
Potential transitive FD (if we did NOT snapshot): billing_id → appointment_id → doctor_id →
consultation_fee This would make consultation_fee transitively dependent on billing_id via doctor_id. If
the doctor raises their fee, historical bills would change – factual inaccuracy! RESOLUTION:
TRG_CREATE_BILLING snapshots the fee at booking time into BILLING. billing_id → consultation_fee
(direct; fee is now an independent historical fact)
```

- Update anomaly avoided: changing doctor fee does not affect past billing records.
- BILLING table is fully in 3NF — no transitive dependencies remain.

Decision 2: CONSULTATION — patient_id NOT stored (preserves 3NF)

```
Potential transitive FD: consultation_id → appointment_id → patient_id Storing patient_id in
CONSULTATION would create: consultation_id → patient_id (via appointment_id – transitive) RESOLUTION:
patient_id NOT stored in CONSULTATION. To find the patient for a consultation: JOIN CONSULTATION →
APPOINTMENT.
```

- Avoids data duplication: patient link is always authoritative from APPOINTMENT.
- Eliminates update anomaly: changing patient assignment only needs one row update.

Decision 3: APPOINTMENT.department_id — documented 3NF trade-off

```
FD that exists in reality: doctor_id → department_id (a doctor belongs to one department) Therefore:
appointment_id → doctor_id → department_id (transitive!) A strict 3NF schema would remove department_id
from APPOINTMENT and always JOIN to DOCTOR. DECISION: department_id KEPT in APPOINTMENT as a documented
denormalization because: (a) Appointments can exist for walk-in/referral paths without a doctor (null
doctor_id) (b) VPD policy function needs department_id at query time without multi-table joins (c)
Explicitly marked with SQL comment; departure from strict 3NF is intentional.
```

Decision 4: LAB_ORDER.patient_id — documented 3NF trade-off

```
Transitive path: lab_order_id → consultation_id → appointment_id → patient_id Storing patient_id
directly in LAB_ORDER creates a transitive dependency. DECISION: patient_id KEPT in LAB_ORDER for: (a)
VPD policy filter: WHERE patient_id = SYS_CONTEXT('clinic_ctx','patient_id') Without it, VPD would need
a 3-level subquery on every lab query. (b) Performance-critical patient lab portal queries. Explicitly
documented in task18_extended_schema.sql comments.
```

3NF Compliance Summary Table

Table	1NF	2NF	3NF	Notes
DEPARTMENT	✓	✓	✓	
DOCTOR	✓	✓	✓	
STAFF	✓	✓	✓	
PATIENT	✓	✓	✓	
MEDICAL_RECORD	✓	✓	✓	
CLINIC_USER	✓	✓	✓	
TOKEN_BLACKLIST	✓	✓	✓	
APPOINTMENT	✓	✓	✓*	department_id is documented denorm
CONSULTATION	✓	✓	✓	patient_id NOT stored — correctly avoids transitive FD
PRESCRIPTION	✓	✓	✓	
PRESCRIPTION_ITEM	✓	✓	✓	Surrogate PK eliminates partial dependency
BILLING	✓	✓	✓	fee snapshotted; total_amount is virtual column
LAB_ORDER	✓	✓	✓*	patient_id is documented denorm for VPD

LAB_RESULT	✓	✓	✓	
VITAL_SIGNS	✓	✓	✓*	patient_id documented denorm for VPD
LAB_TEST_CATALOGUE	✓	✓	✓	
PHARMACY_ITEM	✓	✓	✓	
DISPENSING	✓	✓	✓*	patient_id documented denorm for VPD
DOCTOR_LEAVE	✓	✓	✓	
EMERGENCY_REQUEST	✓	✓	✓	
DOCTOR_NOTIFICATION	✓	✓	✓	
PATIENT_NOTIFICATION	✓	✓	✓	
PATIENT_FAMILY_LINK	✓	✓	✓	
AUDIT_LOG	✓	✓	✓	

5.4 Boyce-Codd Normal Form (BCNF)

Definition: In 3NF AND for every non-trivial functional dependency X → Y, X must be a superkey of the relation.

BCNF Analysis for APPOINTMENT (the most interesting case):

```
APPOINTMENT has two candidate keys: CK1: appointment_id (surrogate PK) CK2: (doctor_id, appt_date,
slot_start) (business key, enforced by UNIQUE constraint) FDs to check: appointment_id → doctor_id,
patient_id, appt_date, slot_start [LHS = superkey ✓] (doctor_id, appt_date, slot_start) →
appointment_id [LHS = superkey ✓] appointment_id → appt_date [LHS = superkey ✓] No non-trivial FD has a
non-superkey LHS → APPOINTMENT is in BCNF.
```

BCNF Analysis for BILLING:

```
BILLING has two candidate keys: CK1: billing_id CK2: appointment_id tax_pct is constrained CHECK
(tax_pct = 18.00) — this is a domain constraint, not an FD. No non-key attribute functionally determines
another non-key attribute. BILLING is in BCNF.
```

Overall BCNF Status: All 24 tables are in BCNF except for the three documented denormalizations (APPOINTMENT.department_id, LAB_ORDER.patient_id, VITAL_SIGNS.patient_id, DISPENSING.patient_id) which violate strict BCNF but are intentional and documented trade-offs for VPD filter performance. No unintentional BCNF violations exist in the schema.

6. Trigger Catalogue (9 Triggers — Minimum Required: 2)

The project implements 9 triggers — 7 more than the minimum. At least one enforces a business rule and at least one automates a state change.

Trigger Name	Event	Purpose	Type
TRG_CREATE_BILLING	AFTER INSERT on APPOINTMENT	Auto-creates a BILLING row with snapshotted consultation fee and GST = 1	State automation
TRG_BILLING_STATUS	BEFORE UPDATE on BILLING	Automatically sets payment_status = PENDING / PARTIAL / PAID based on	State automation
TRG_AUDIT_PATIENT	AFTER INS/UPD/DEL on PATIENT	Writes JSON old/new value diff into AUDIT_LOG — immutable audit trail.	Audit
trg_dispensing_deduct_stock	AFTER INSERT on DISPENSING	Decrements PHARMACY_ITEM.stock_quantity by quantity_dispensed. Prev	Business rule
trg_doctor_leave_block_booking	BEFORE INSERT on APPOINTMENT	Raises ORA-20010 if the proposed appointment overlaps an APPROVED D	Business rule
trg_emergency_fanout	AFTER INSERT on EMERGENCY_REQUEST	Notifies ALL doctors for CRITICAL severity; notifies assigned department do	Notification
trg_cancel_voids_invoice	AFTER UPDATE on APPOINTMENT	Sets BILLING.payment_status = VOID when appointment is cancelled. Keep	State automation
trg_cancel_notify_doctor	AFTER UPDATE on APPOINTMENT	Inserts a CANCELLATION notification into DOCTOR_NOTIFICATION when	Notification
trg_lab_result_notify	AFTER INSERT on LAB_RESULT	Inserts a LAB_READY notification into PATIENT_NOTIFICATION when lab	Notification

7. Virtual Private Database (VPD) Design ★ BONUS

Oracle's DBMS_RLS (Row-Level Security) is used to enforce row-level access control at the database layer — completely transparent to the application. This is a **graduate-level Oracle feature** not required by the problem statement.

Context Package — Session Variables

```
clinic_ctx_pkg.set_role(role) -- 'ADMIN' | 'DOCTOR' | 'PATIENT' clinic_ctx_pkg.set_doctor_id(id) --
called on every doctor login clinic_ctx_pkg.set_patient_id(id) -- called on every patient login
```

Role	Table	Predicate Appended by Oracle
ADMIN	All protected tables	(no restriction — sees all rows)
DOCTOR	APPOINTMENT	doctor_id = SYS_CONTEXT('clinic_ctx', 'doctor_id')
DOCTOR	CONSULTATION	appointment_id IN (SELECT appt_id FROM APPOINTMENT WHERE doctor_id = ...)
DOCTOR	LAB_ORDER	ordered_by_doctor_id = SYS_CONTEXT('clinic_ctx', 'doctor_id')
PATIENT	APPOINTMENT	patient_id = SYS_CONTEXT('clinic_ctx', 'patient_id')
PATIENT	MEDICAL_RECORD	patient_id = SYS_CONTEXT('clinic_ctx', 'patient_id')
PATIENT	CONSULTATION	appointment_id IN (SELECT appt_id FROM APPOINTMENT WHERE patient_id = ...)
PATIENT	LAB_ORDER, VITAL_SIGNS, DISPENSING	patient_id = SYS_CONTEXT('clinic_ctx', 'patient_id')
Unknown	**All tables**	**1=0 — zero rows visible**

Protected Tables: APPOINTMENT, BILLING, CONSULTATION, PRESCRIPTION, MEDICAL_RECORD, LAB_ORDER, LAB_RESULT, VITAL_SIGNS, DISPENSING (9 tables).

8. Authentication Architecture ★ BONUS

Portal	Method	Tokens	Logout
Staff / Admin / Doctor	Username + bcrypt password hash stored in USER_USER	JWT access token (30 min) + refresh token (7 days); Oracle session context set via set_role()	Access token added to TOKEN_BLACKLIST
Patient	Phone number → 6-digit OTP (hashed with bcrypt, TTL 10 min)	JWT access token (30 min) + refresh token (7 days); Oracle session context: set_role('PATIENT'), set_patient_id()	Same TOKEN_BLACKLIST mechanism

9. API Route Summary

Patient Portal /api/patient/

Method	Path	Description
GET	`/profile`	Own profile
PUT	`/profile/complete`	Complete registration ✓ (required)
GET	`/appointments`	Own appointments (VPD-scoped)
POST	`/appointments`	Book appointment ✓ (required)
PATCH	`/appointments/{id}/cancel`	Cancel scheduled appointment
GET	`/doctors`	List available doctors
GET	`/medical-records`	Health summary (VPD-scoped)
GET	`/consultations`	Full consultation history timeline
GET	`/prescriptions`	Prescriptions with medication items
GET	`/notifications`	In-app notifications
PATCH	`/notifications/read-all`	Mark all read
POST	`/emergency`	Raise emergency request ★

Doctor Portal /api/doctor/

Method	Path	Description
GET	`/appointments`	VPD-scoped schedule
GET	`/patients`	Distinct patients seen
POST	`/appointments/{id}/consultation`	Create consultation notes
POST	`/appointments/{id}/prescription`	Issue prescription ✓ (required)
PATCH	`/appointments/{id}/complete`	Mark appointment complete
GET	`/leave`	Own leave requests ★
POST	`/leave`	Request leave ★
GET	`/notifications`	In-app notifications ★
PATCH	`/notifications/read-all`	Mark all read ★

Admin Portal /api/admin/

Method	Path	Description
GET	`/dashboard-stats`	KPI cards (patients, appointments, revenue, pending bills) ✓
GET	`/revenue-report`	Per-department revenue — refreshes materialized view ✓ (required)
GET	`/emergency-queue`	Severity-sorted emergency requests ★
PATCH	`/emergency/{id}/assign`	Assign doctor to emergency ★
PATCH	`/emergency/{id}/resolve`	Resolve emergency ★

10. Views, Materialized Views & Indexes

10.1 Views & Materialized View

View Name	Type	Description
V_PATIENT_SUMMARY	Regular View	Patient ID, name, age, blood group, last appointment date, total visits, outstanding b
V_DOCTOR_SCHEDULE_TODAY	Regular View	All today's slots for all doctors with patient name and appointment status.
V_BILLING_OUTSTANDING	Regular View	Unpaid and partially-paid invoices with full patient and doctor details.
V_DEPT_REVENUE	Materialized View ★	Per-department monthly revenue: total billed, total collected, collection rate %. Refre

10.2 Index Catalogue (13 Indexes)

Index Name	Table	Column(s)	Rationale
idx_appt_patient	APPOINTMENT	patient_id	Patient portal appointment queries + VPD predicate
idx_appt_doctor	APPOINTMENT	doctor_id	Doctor schedule queries + VPD filter
idx_appt_date	APPOINTMENT	appt_date	Date-range filters for scheduling calendar
idx_appt_dept	APPOINTMENT	department_id	Department-based appointment reports
idx_billing_appt	BILLING	appointment_id	Billing lookup by appointment
idx_lab_order_patient	LAB_ORDER	patient_id	Patient lab results query
idx_lab_order_doctor	LAB_ORDER	ordered_by_doctor_id	Doctor's ordered tests view
idx_lab_order_consult	LAB_ORDER	consultation_id	All orders per consultation
idx_lab_order_status	LAB_ORDER	status	Admin lab management status filters
idx_vital_patient	VITAL_SIGNS	patient_id	Patient vitals history
idx_dispensing_patient	DISPENSING	patient_id	Pharmacy dispensing per patient
idx_dispensing_presc	DISPENSING	prescription_id	Dispensing per prescription
idx_pharmacy_active	PHARMACY_ITEM	is_active, is_deleted	Active stock query filter

11. Bonus Features Summary ★

The following features go significantly beyond what was required by the problem statement. They demonstrate advanced Oracle capabilities and production-grade application design:

★ Virtual Private Database (VPD) *DBMS_RLS*

Row-level security enforced at the Oracle database layer using DBMS_RLS. Policy functions dynamically append WHERE predicates based on the session role/ID. Applied to 9 tables. Completely transparent to the FastAPI application — even if the application is bypassed, data isolation is preserved at the DB layer.

★ OTP Authentication for Patients *FastAPI + Oracle*

Patients authenticate via phone number → 6-digit OTP (stored hashed via bcrypt, TTL 10 minutes). No passwords required for patients. On verify, a JWT is issued.

★ JWT Token Blacklist *Oracle Table*

TOKEN_BLACKLIST table stores revoked JWT hashes. Middleware checks every request against this table — prevents token reuse after logout.

★ Materialized View with On-Demand Refresh *Oracle DBMS_MVIEW*

V_DEPT_REVENUE materializes per-department revenue aggregations. Refreshed via DBMS_MVIEW.REFRESH when the admin calls the revenue-report endpoint. Powers the admin revenue chart without expensive live aggregations.

★ AUDIT_LOG with JSON CLOBs *Oracle Trigger*

TRG_AUDIT_PATIENT captures every INSERT/UPDATE/DELETE on the PATIENT table, storing old and new values as JSON CLOBs in AUDIT_LOG — immutable audit trail.

★ Lab Test Module (Task 18) *4 new tables*

Full lab workflow: LAB_TEST_CATALOGUE (20 tests), LAB_ORDER (doctor raises test), LAB_RESULT (result + is_abnormal flag), VITAL_SIGNS (pre-consult BP/HR/SpO■/temp/weight/height). Includes trg_lab_result_notify trigger for patient alerts.

★ Pharmacy & Dispensing (Task 18) *2 new tables + trigger*

PHARMACY_ITEM drug inventory with batch/expiry/reorder threshold. DISPENSING table + trg_dispensing_deduct_stock trigger automatically decrements stock on every dispense.

★ Emergency System (Task 19) *EMERGENCY_REQUEST + trigger*

Patients can raise emergencies with severity levels (LOW/MODERATE/HIGH/CRITICAL). trg_emergency_fanout broadcasts notifications: ALL doctors for CRITICAL; department doctors otherwise. Admin can assign and resolve via dedicated API.

★ Doctor Leave & Booking Guard (Task 19) *DOCTOR_LEAVE + trigger*

Doctors can declare leave windows. trg_doctor_leave_block_booking raises ORA-20010 on any booking attempt that overlaps an APPROVED leave window.

★ Bidirectional Family Links (Task 19) *PATIENT_FAMILY_LINK*

M:N self-relationship on PATIENT for 5 relationship types: SPOUSE, PARENT, CHILD, SIBLING, GUARDIAN. Useful for family insurance and emergency contact lookups.

★ In-App Notification System (Task 19) *2 tables + triggers*

DOCTOR_NOTIFICATION and PATIENT_NOTIFICATION tables with is_read flag. Fed by trg_emergency_fanout, trg_cancel_notify_doctor, trg_lab_result_notify.

★ Virtual Computed Column (BILLING) *Oracle Feature*

BILLING.total_amount is declared as a VIRTUAL column: (consultation_fee + additional_charges) * (1 - discount_pct/100) * (1 + tax_pct/100). Oracle computes it on every read — no storage, always consistent.

Submitted by: Group 23 | bt23cse012 · bt23cse002 · bt23cse007

Subject: Database Management Systems (S26)

Stack: Oracle 19c · FastAPI · React + TypeScript